

LoadLead Platform E2E Audit v7

Date: 2026-07-16 · **Scope:** backend (Express/TS/DynamoDB), frontend-v2 (React/Vite), infra (Terraform/OpenTofu), across dev / staging / prod. Six parallel dimensions: build+test health, business-logic correctness, security/IAM, environment parity + infra, live prod/staging smoke, and a deploy-delta ("not yet in prod") register. Baseline: `origin/main` HEAD `42bc406` ; last prod deploy `20b54cc` (H9 phase 5).

1. Executive summary

The **v6 remediation is real and complete**. Every one of the v6 **3 CRITICAL** and **13 HIGH** findings was verified fixed in current `main` by direct source read (Section 3). Money and legal invariants are clean (integer cents, append-only ledgers, first-accept-wins, tenant isolation). The suite is **890/890 green** (up from 772), frontend builds clean with the known-good chunk shape, both parity CI gates pass, and prod is healthy, hardened, and private (POD bucket locked down this session).

The risk this round was **one new authorization hole and a CI blind spot - both now closed**. The adversarial pass found **1 new HIGH bordering CRITICAL**: a driver could self-declare `ownedByOperatorId` via an unvalidated profile update and inherit any VERIFIED carrier's FMCSA authority + insurance without consent (N1). One **MEDIUM** was a latent parity-checker gap that silently excluded the platform's single most sensitive table (ENV-1). **Both were fixed and deployed to prod on 2026-07-17** (PR #106, backend bundle `loadlead-backend-20260717081549`) - see Section 2 and the remediation status below. The remaining items are LOW/hygiene or intentionally-gated (M1 AML inert; Canopy prod-disabled).

Remediation status (post-audit)

ID	Severity	Status
N1	HIGH	FIXED + deployed - PR #106, bundle <code>loadlead-backend-20260717081549</code> . Verified live: attack surface 401-gated; prod pre/post-check shows 0 drivers lost carrier authority .
ENV-1	MEDIUM	FIXED - PR #106. Checker now reports 54 prefix-derived slots (was 53) and still exits 0; the W9/TIN access log is inside the guard.
N2 (M1 AML)	MEDIUM	OPEN, blocked on enabling Didit's standalone AML product (Section 5).
N3-N6, INF-1..8	LOW	OPEN - batchable hygiene (COA-3/COA-4).

Findings by severity (this round)

Severity	Count	Theme
CRITICAL	0	- (all 3 v6 CRITICALs remediated)
HIGH	1 (fixed)	N1: driver self-affiliation mass-assignment → carrier-authority bypass
MEDIUM	2 (1 fixed, 1 open)	ENV-1 (fixed): parity checker blind to W9-access-log table; N2 (open): AML enforcement inert (blocked on Didit)
LOW	~12	accessorial policy-accept ungated; org-invite privileged role (not exploitable); JWT base-role TTL; loadboard full-scan; Lambda node20 EOL; stale <code>imported-tables.tf.draft</code> ; orphan <code>LoadLead_AdminAudit</code> ; Canopy secrets out-of-band; dead staging tfvars; dead <code>/api/health</code> handlers; dev enumerates tables vs prefix; BOL GSI drift

Delta from v6

- **Closed since v6:** all 3 CRITICAL (C1 pagination, C2 self-signup ADMIN, C3 org takeover), all 13 HIGH (IDOR cluster H1-H5, maps H6, Didit fail-open H7, hot-path scans H8, uploads/public-bucket H9 [fully, phases 2-5 this session], admin tier H10, deps H11, SNS tests H12, dev capacity table H13), plus MEDIUMs M2/M3/M4/M9/M11 and the LOW hygiene batch. **H9 is now the strongest area** (private bucket, signed serving, size-capped uploads, DoD matrix).
- **Still open by design:** M1 AML (wired inert, activation blocked on Didit - see N2/Section 5).
- **New this round:** N1 (HIGH) and ENV-1 (MEDIUM) - both since fixed and deployed - plus the LOW/infra cluster (open).

2. Severity-ranked findings

HIGH

N1 - Driver self-affiliation via mass-assignment → carrier-authority (Gate 1) bypass. `PUT /api/driver/profile`

(`routes/driver.ts:83`) has no validation schema and passes raw `req.body` to `DriverService.updateProfile` (`driverService.ts:177`), which spreads `...updates` and strips only `headshotUrl`. `createProfile` also copies `ownedByOperatorId` from the body (`driverService.ts:107`). `resolveCarrierOfRecord` (`carrierOfRecord.ts:39-46`) then trusts `driver.ownedByOperatorId` **with no back-check** that the operator's fleet includes this driver.

- **Failure scenario:** an attacker signs up as DRIVER, sets `{"ownedByOperatorId":"oo_<a known VERIFIED operator>"}` on their profile, completes their own genuine IDV (Gate 2 checks only the attacker's `user.idvStatus`), and `requireVerifiedCarrier` passes - Gate 1 resolves the victim operator as VERIFIED. The attacker accepts/hauls real freight under a legitimate carrier's FMCSA authority and insurance without consent, defeating the core "only a VERIFIED carrier may accept loads" invariant. Operator ids leak via offers, dashboards, NOA docs, and `acct:${carrierId}` payee strings.
- **Verified** against source (the four hops above were all present and unguarded).
- **STATUS: FIXED + deployed 2026-07-17** (PR #106, bundle `loadlead-backend-20260717081549`). Two layers, since either alone leaves a gap:
 1. `routes/driver.ts` strips server-controlled fields (`driverId` , `userId` , `ownedByOperatorId` , `isSelf` , `status`) on POST + PUT `/profile` . At the ROUTE, not the service, so the server-derived writers (self-driver creation, the consented fleet-invite accept, `createOrgDriver` , `updateStatus`) are unaffected.
 2. `resolveCarrierOfRecord` requires the operator to corroborate the link: the driver is in its `fleetDriverIds` , OR the driver row belongs to the operator's own `userId` . `driver.isSelf` is deliberately not consulted - it is forgeable by the same vector. An unclaimed link falls through to the org path, so a legitimate org driver with a stale field still resolves their real carrier. `isFleetCarrierDriver` 's raw-field shortcut was collapsed into the same resolver (it could otherwise disagree with it about the same driver).
- **Two deliberate departures from the original recommendation**, both validated against real data:
 - `carrierId` is NOT stripped: it is a user-entered, required profile field (`REQUIRED_PROFILE` in `SettingsPage`), so stripping it would make the driver profile impossible to complete. It is not an authority field - `resolveCarrierOfRecord` never reads it.
 - The back-check is not `fleetDriverIds.includes()` alone: the OO self-driver is deliberately never in that list. A prod pre-check found the ONE linked driver in prod is exactly that self-driver (`in_fleetDriverIds=false` , `same_userId=true`) - the naive check would have silently broken OO self-haul in production.
- **Rollout proof:** prod pre- and post-deploy checks both report **0 drivers would lose OO authority**; live probes return 401 on the attack surface; health 200 `productionHardened` .
- Regression tests: 4 resolver-refusal cases + 3 route-strip cases + 1 fleet-pool case (suite 890 -> 898).

MEDIUM

ENV-1 - Parity checker is blind to the W9-access-log table (regex excludes digits). `scripts/check-table-env-parity.mjs:41-43` matches `DYNAMODB_[A-Z]_*_TABLE` ; the character class excludes digits, so `DYNAMODB_W9_ACCESS_LOG_TABLE` never matches. The checker verifies only 53 of 54 config slots and **never confirms dev/staging override the W9 table** - the append-only log of every full-W9/TIN open, which `environment.ts:166` calls "the most sensitive read on the platform." Masked today (dev enumerates it at `dev/main.tf:243` ; staging derives it from the prefix), but a future edit dropping the dev line would leave CI green while dev silently

resolved the prod `LoadLead_W9AccessLog` ; only the runtime boot guard would catch it. **STATUS: FIXED** (PR #106) - all three regexes are now `DYNAMODB_[A-Z0-9_]+_TABLE` ; the checker reports 54 prefix-derived slots (was 53) and still exits 0, so the W9/TIN access log is finally inside the guard rather than merely appearing to be.

N2 - AML enforcement ships inert (carryover of v6 M1). `deriveStatus` treats `amlStatus === undefined` as passing while `AML_REQUIRED` is off (`verification.ts:98,110`). Wiring (#97) + runbook (#104) landed, but the control is not enforced in prod today. **Activation is blocked on Didit** (see Section 5). **Fix:** enable the standalone Didit AML product (or switch to workflow-sourced AML), backfill, then `AML_REQUIRED=true` + a boot assertion.

LOW (condensed; full detail in dimension files)

ID	Finding	Where
N3	Accessorial policy-accept ungated by load party - any carrier can e-sign a policy acceptance for any <code>loadId</code> (integrity/spam, not money)	<code>routes/accessorials.ts:210-241</code>
N4	Org invite validator accepts privileged <code>userRole</code> - not exploitable (<code>membership.userRole</code> is never an authz source; staff-accept requires <code>invite.platformRole</code>) but tighten it	<code>routes/org.ts:537</code>
N5	Base <code>role</code> trusted from a 7-day JWT with no revocation (sensitive tiers correctly re-derive from DB)	<code>middleware/auth.ts:54</code>
N6	Loadboard scans full lock/offer/negotiation tables per poll (correct after pagination fix, worse cost profile) - deferred pending GSIs	<code>negotiationService.ts:133, 279, 736</code>
INF-1	<code>loadlead-prod-signatures-worm-sink</code> runs <code>nodejs20.x</code> (EOL) - bump to <code>nodejs22.x</code> (safe; handler uses only stable APIs)	<code>envs/prod/worm-sink.tf:196</code>
INF-2	Stale drifted <code>imported-tables.tf.draft</code> re-declares imported tables + lacks the M6 GSI - delete it	<code>envs/prod/imported-tables.tf.draft</code>
INF-3	Orphaned prod table <code>LoadLead_AdminAudit</code> (no backend reader; app uses <code>LoadLead_AdminAuditLog</code>)	<code>envs/prod/imported-tables.tf:416</code>
INF-4	Canopy <code>CLIENT_ID/SECRET/WEBHOOK_SECRET</code> set out-of-band on EB, not in tfvars - an env recreate wipes them	<code>envs/staging/main.tf:279</code> , <code>envs/prod/canopy.tf:54</code>
INF-5	Dead <code>NODE_ENV</code> / <code>BETA_MODE</code> values in <code>staging.auto.tfvars</code> (overridden by <code>merge()</code>)	<code>staging.auto.tfvars:9-10</code>
INF-6	Dead duplicate <code>/api/health</code> handlers contradict the "consolidated to one place" comment	<code>backend/src/index.ts:425-431</code>
INF-7	Dev enumerates ~64 table vars instead of <code>DYNAMODB_TABLE_PREFIX</code> (fragile; new tables must be hand-added)	<code>envs/dev/main.tf:177-251</code>
INF-8	BOL GSI drift - prod <code>LoadLead_BOL</code> has a <code>status-index</code> the dev/staging tableset lacks (query-by-status would 500 non-prod)	<code>imported-tables.tf:337</code> vs <code>dynamodb-tableset/main.tf:118</code>

3. v6 CRITICAL / HIGH - confirmed remediated in current main

v6 ID	Finding	Evidence
C1	DB scan/query 1 MB truncation	<code>config/database.ts:108-119</code> (query) + <code>:147-151</code> (scan) loop on <code>LastEvaluatedKey</code>
C2	Self-signup ADMIN	<code>utils/validators.ts:19</code> (<code>isIn(SELF_SIGNUP_ROLES)</code>) + <code>authService.ts:70</code> allowlist + <code>platformRole.ts:69</code> (<code>resolvePlatformRole(null)→null</code>) + <code>routes/auth.ts:132</code> ADMIN MFA gate
C3	Cross-tenant org takeover	<code>routes/org.ts:311</code> <code>resolveMembershipInOrg</code> on PATCH/DELETE/suspend/reinstate + <code>orgService.ts</code> <code>expectedOrgId</code> guards
H1	Accessorial charge lifecycle IDOR	<code>accessorials.ts:263</code> <code>assertCallerIsLoadShipper</code>
H2	Factoring package/export cross-tenant	<code>factoring.ts:402,426</code> <code>assertCallerActsForLoad</code> ; carrierId server-derived
H3	Receiver reads any load	<code>receiver.ts:43</code> <code>load.receiverId === receiver.receiverId</code> else 404
H4	BOL creation on another shipper's load	<code>bol.ts:74</code> shipper-owns-load; reads via <code>requireBOLAccess</code>
H5	Org suspend/reinstate tenant-binding	same <code>resolveMembershipInOrg</code> path
H6	<code>/api/maps/*</code> unauth + unthrottled	<code>maps.ts:13</code> <code>authenticate</code> + <code>index.ts</code> <code>mapsRateLimiter</code>
H7	Didit webhook fail-open	<code>verification.ts:403</code> 401 in prod when secret unset
H8	Hot-path profile scans	driver/shipper/receiver services now <code>queryIndexOrScan</code> (userId-index, REQUIRED boot guard)
H9	Presigned uploads (MIME/size/ownership/public bucket)	MIME allowlist (415) + POD assigned-driver check + size-capped <code>presignedPodPost</code> + private serving + PAB all-true (this session, phases 2-5)
H10	Admin grant/revoke on bare requireAdmin	<code>admin.ts:82,90</code> now <code>requireStaffTier(DESTRUCTIVE_TIER)</code>
H11-H13	deps / SNS tests / dev capacity table	axios bumped; snsVerify covered; <code>dev/main.tf:184</code> capacity slot pinned

Money & correctness invariants checked clean: integer cents only (`utils/money.ts` , `assertIntegerCents` in accessorial + payout-intercept); append-only ledgers (charge history append; terminal SETTLED; intercepts never mutate the invoice, per-(intercept,invoice) idempotency key); first-accept-wins negotiation lock (`attribute_not_exists(loadId)` conditional put); factoring carrierId server-derived.

4. AWS operational notices (from account emails)

Notice	Impact	Action
Lambda Node.js 20.x EOL - no updates after 2027-03-03	1 function: <code>loadlead-prod-signatures-worm-sink (nodejs20.x)</code> ; staging toggle already <code>nodejs22.x</code>	INF-1: bump <code>worm-sink.tf:196</code> to <code>nodejs22.x</code> , re-zip, apply. Prod-only PR (<code>[terraform-prod-only]</code>). Not urgent.
Billing console CloudTrail event rename (Sept 28-Oct 7 2026)	Zero - nothing in LoadLead matches <code>GetPaymentPreference / billingconsole .amazonaws.com / Preferences_*</code>	None. Informational.

5. Not-yet-in-production register

- **Bucket A (merged to main, not deployed): NONE.** Prod backend + frontend are current with main's code. The only delta `20b54cc..42bc406` is the docs-only M1 runbook (#104).
- **Bucket B (un-landed): only 2 open PRs, both docs/tracking-only - #105** (M1 runbook command fix, mergeable) and **#83** (v6 report + jira sync; report files already in main via #92, only `jira/sync.py` +manifest genuinely un-landed). **No un-landed application code anywhere.** ~90 "ahead" branches are squash-merge artifacts (content already in main); 3 `test/*` + `verify/*` branches are throwaway.
- **Bucket C (gated / intentionally not-live):**
 - **M1 AML enforcement** - deployed inert; `AML_REQUIRED` unset. **Blocked on Didit:** the backend calls the standalone `/v3/am1/` API which returns 403 ("no permission"); AML exists on the Didit account only as a *workflow* feature (published "Custom KYB" workflow), not the standalone product. Flipping now would 403-block all new onboarding. Enable the standalone AML product (or switch to workflow-sourced AML), backfill, then flip.
 - **Canopy Connect** - code is in the deployed prod bundle but `connectEnabled=false` in prod (no prod Canopy creds; sandbox-first). Live + verified on staging. Open blockers: webhook-signature confirmation, SDK URL, Components-plan access.
 - **Compliance carrier-documents** - fully live in prod; minor deferred UI (hauler policy-sign button, sidebar links, `CarrierComplianceView` in shipper Load Detail).

6. Live smoke (prod + staging)

Prod: `/api/health` 200 `productionHardened:true` ; frontend 200; 13/13 unauthenticated route probes return 401 (no 5xx, no leak); EB Ready/Green on `loadlead-backend-20260716202247` ; `AML_REQUIRED` unset (OFF); POD bucket PAB all-true + raw public GET 403 (privatization holding). Staging: frontend 200, API paused (expected - ASG idle behind the toggle). No failures.

7. Course of action (prioritized)

COA-1 - the one live authorization hole: **DONE (2026-07-17, PR #106).**

- **N1:** route-level strip of server-controlled fields + operator corroboration in `resolveCarrierOfRecord` + 8 regression tests. Deployed and verified in prod; 0 drivers lost authority.
- COA-2 - near-term (CI trust + AML):
- **ENV-1: DONE** (PR #106) - parity-checker regex now includes digits; config count 54.
 - **N2 / M1 AML: OPEN, blocked.** Enable the standalone Didit AML product (the account has AML only as a workflow feature, so the backend's `/v3/am1/` call 403s), backfill the verified entities, then flip `AML_REQUIRED=true` + boot assertion. Flipping before that would 403-block all new onboarding. (User-gated; see `docs/M1-AML-ACTIVATION-RUNBOOK.md` .)

COA-3 - hardening + hygiene (batchable):

- N3 (gate accessorial policy-accept on offer/claim eligibility), N4 (tighten org-invite role validator), N6 (loadboard GSIs).
- INF-1 (Lambda node22 bump), INF-2 (delete `imported-tables.tf.draft`), INF-3 (remove/annotate `LoadLead_AdminAudit`), INF-4 (Canopy secrets into tfvars), INF-5/INF-6 (dead tfvars values / dead health handlers), INF-8 (add BOL `status-index` to the tableset module).

COA-4 - larger refactors (schedule):

- INF-7: migrate dev to `DYNAMODB_TABLE_PREFIX` (retire the ~64-line enumeration).
- N5: JWT tokenVersion/revocation for base role.

8. Confidence - verified clean this round

890/890 backend tests green (0 flake); frontend clean + known-good chunk shape (no react-vendor split); both parity CI gates pass; all 3 v6 CRITICAL + 13 HIGH remediated; money primitives (integer cents, append-only, idempotent intercepts); negotiation first-accept-wins; tenant isolation on factoring/charges/BOL/receiver/org; POD/headshot privatization holding in prod (PAB all-true, 403 on public reads); prod healthy + hardened; deploy state fully reconciled (prod == main code).

9. Appendix - dimensions

Six parallel passes: build+test health, business-logic, security/IAM (merged here as Sections 2-3), environment parity + infra, live smoke, and the deploy-delta register. This report is the consolidated synthesis; per-dimension agent transcripts are in the session task logs.